Clase 15 · Ciclo de vida de procesos y tareas de tiempo real

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · lunes 19 oct 2026 · Tema 3 (3.2–3.3) · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Segunda clase de la Unidad 3 (entra en el segundo parcial). Al inicio se devolvieron los exámenes del primer parcial. Continúa la Clase 13.

3.2.1 Instanciación de un proceso

Definición

Instanciar es cargar en RAM el identificador y la estructura de administración del programa; lo hace el sistema operativo antes de poder ejecutarlo.

1. El SO crea una **estructura de proceso vacía** (PID, banderas, campos de administración).
2. **Lee el encabezado del archivo** antes de reservar memoria, por seguridad: puede ser un ejecutable de esa plataforma, un script con shebang (`#!/bin/bash`), una clase de Java, un binario de otra arquitectura… o una imagen GIF.
3. Si es un ejecutable válido, del encabezado lee si es de **16, 32 o 64 bits** y cuánto pide.
4. Dimensiona la estructura, solicita la RAM inicial y arma el **pool de memoria** (heap y variables).

Así el sistema no interpreta datos como instrucciones de CPU. Error común: creer que el SO copia el ejecutable a RAM "a ciegas".

3.2.2 Los tres estados base

Estado	Qué significa
WAIT	Recién creado: el programa todavía se copia de disco a la memoria y se registran sus páginas
READY	Cargado y listo, formado en la cola de planificación (p. ej. Round Robin); aún no ejecuta
RUNNING	En el núcleo; **solo uno por núcleo** a la vez

- Si en RUNNING el hilo llama a `fork`, se duplican los segmentos, se crea otra entrada en la tabla de procesos con PID propio y el hijo entra en **READY**.
- Al agotarse el **cuanto** (time slice), una interrupción del temporizador despierta al SO, que regresa el proceso de RUNNING a READY. Así se logra la multitarea (system tick, interrupciones de la Unidad 2).

Error común: creer que en READY ya se ejecuta código.

3.2.3 Bloqueo por E/S y la regla de retorno

Si un proceso en RUNNING pide datos a un periférico (`scanf()`, puerto serie, red), en vez de hacer sondeo activo (polling) el SO lo pasa a **WAIT/BLOCKED**: no consume CPU y otro proceso READY usa el núcleo.

> **Regla de examen**
> Cuando la E/S termina (interrupción de hardware), el proceso **nunca pasa directo de WAIT a RUNNING**: siempre vuelve a **READY** y espera su turno. (Al presionar Enter en un `scanf()` el programa no se ejecuta de inmediato.)

3.2.4 Estados extendidos

Estado	Significado	Ejemplo
PENDING	Cargado, pero programado para un momento dado	Scripts de `cron`/`crontab` (respaldos, `apt update` de madrugada)
DELAYED	Listo, pero pospuesto por uno de mayor prioridad	Respaldo que cede ante una consulta a la base de datos
SUSPENDED	Congelado; **0 % de CPU**	`SIGSTOP`, Ctrl+Z, pausar VLC
STOPPING	Transición de cierre: limpieza y destructores	Cerrar conexiones, liberar cámara o puerto serie
STOPPED / KILLED	Terminó; recursos liberados de la RAM	—

Error común: confundir suspender (0 % CPU) con minimizar una ventana (sigue ejecutando).

3.2.5 Procesos e hilos (GPOS) frente a tareas (RTOS)

	GPOS (Linux, Windows)	RTOS (FreeRTOS, VxWorks)
Unidad mínima	**Hilo** (thread); todo proceso nace con un hilo principal	**Tarea** (task)
Memoria	Procesos aislados por la MMU; los hilos de un proceso comparten memoria virtual, datos y privilegios ("procesos ligeros")	Todas las tareas en un **superproceso raíz** (RT process); memoria compartida o verificada en compilación por el linker
Carga	Loader dinámico	Sin loader dinámico: todo enlazado desde la compilación
Planificación	Equidad (Round Robin)	**Prioridad fija** y **deadline** estricto

GPOS para aplicaciones complejas con memoria virtual; RTOS para responder en microsegundos sin sobrecarga.

3.3 Tareas de tiempo real

En tiempo real se descarta Round Robin porque no es determinista. Bucle clásico de control (cada etapa espera a la anterior):

Sensado → Filtrado → Ley de control → Actuación

Tipo	Cómo ocurre	Ejemplo / papel
Periódica	Intervalos fijos (periodo T), tiempo de ejecución y deadline	Base del análisis de **planificabilidad** (schedulability)
Aperiódica	Sin periodo, pero con deadline obligatorio e inflexible	Frenado o bolsas de aire de un vehículo autónomo
Esporádica	Momentos no determinados, con **separación mínima garantizada** entre ocurrencias	Variante de la aperiódica

> **Tiempo real ≠ rápido**
> "Tiempo real" significa **determinismo**: cumplir los plazos siempre. Fallar el tiempo equivale a fallar el sistema físico.

Avisos

- Próxima clase: tipos de scheduling (3.3), que luego se pospuso (Clase 16). La Raspberry Pi del laboratorio debe correr Linux en consola. Sin tarea nueva.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Instanciar. Estructura vacía → leer encabezado (¿ejecutable?, ¿16/32/64 bits?) → reservar RAM y pool.

> **Estados (examen)**
>
> WAIT (cargando) → READY (formado) → RUNNING (en el núcleo, uno por núcleo). Fin del cuanto: RUNNING → READY. Bloqueo por E/S: RUNNING → WAIT → **READY**, nunca directo a RUNNING.

Extendido	Clave
PENDING	Programado (cron)
DELAYED	Cede ante mayor prioridad
SUSPENDED	SIGSTOP / Ctrl+Z, 0 % CPU
STOPPING	Limpieza y liberar periféricos
STOPPED	Recursos liberados

GPOS vs. RTOS. GPOS: procesos aislados (MMU) e hilos que comparten memoria; loader dinámico. RTOS: tareas en un superproceso raíz, prioridad fija, deadline, todo enlazado al compilar.

Tareas de tiempo real. Periódicas (base del análisis), aperiódicas (deadline inflexible: bolsas de aire), esporádicas (separación mínima garantizada). Bucle: sensado → filtrado → ley de control → actuación.